A. Serialization choices

Four serialization formats are used in the system, each chosen for the constraints of its specific link.

YAML for configuration (sensors.yaml). Configuration is edited by a human operator, so human-readability and the ability to include comments are essential. YAML's lightweight syntax (no closing tags) and support for native data types (lists, maps) reduce editing errors compared to XML or JSON, while its verbosity is not a problem because the configuration file is small and read only at startup.

Protobuf on the sensor-to-server link. Sensors transmit frequent readings (often every few seconds) over TCP. Protobuf produces a compact binary encoding with very low serialisation/deserialisation overhead, which reduces bandwidth and CPU usage – critical for a low-cost agricultural deployment where sensors may run on constrained microcontrollers. The required .proto schema also enforces a contract between sensor and server, catching type mismatches early.

JSON and XML on the public REST API. The API must serve diverse clients: a mobile app prefers JSON for its native parsing speed and low overhead; a legacy desktop tool may only support XML. JSON is the de facto standard for web/mobile clients, while XML offers broader compatibility with older enterprise systems. Both are text-based, debuggable, and well supported by HTTP libraries – a reasonable trade-off because historical queries are infrequent compared to sensor ingress, so the larger message size is acceptable.

B. Web services protocol

The historical data API is designed as a RESTful web service because the problem domain is inherently resource-oriented: sensors and readings are resources that can be identified by URLs (e.g., /sensors/{id}/readings). REST exposes a uniform interface (GET, POST, DELETE) which matches exactly the required operations – listing, adding, removing sensors, and querying readings.

REST is stateless: each request from a client contains all necessary information (sensor ID, time range, Accept header). This simplifies server implementation, improves scalability (no session affinity needed), and lever-ages HTTP caching for repeated queries. In contrast, SOAP would be overkill: it requires XML envelopes, WSDL contracts, and adds unnecessary complexity for a simple data retrieval API. An RPC style (e.g., gRPC) is possible, but it would force clients to use a specific library and binary format, breaking interoperability with the legacy desktop tool that only speaks XML/HTTP. REST, combined with content negotiation, gives the highest interoperability with the least coupling.

C. AsyncIO vs threading

The system is heavily I/O-bound: sensors push data over TCP, the REST API waits on database queries, and the WebSocket feed broadcasts messages to slow clients. In such a workload, most time is spent waiting for network data or disk I/O, not on CPU-bound computations.

AsyncIO uses an event loop that runs many coroutines concurrently on a single operating system thread. When a coroutine performs an await (e.g., waiting for a socket read), the event loop switches to another ready coroutine. This achieves very high concurrency with minimal memory overhead – tens of thousands of connections can be handled using only a few hundred KB per coroutine.

Threads (even with a thread pool) would be less efficient: each thread requires a separate stack (often 1–8 MB) and OS scheduling overhead. Context switches between threads are more expensive than

coroutine yield points. Moreover, shared state (e.g., the broadcaster's client list) would need explicit locks, raising the risk of deadlocks or race conditions. AsyncIO's single-threaded model avoids locks entirely: as long as coroutines yield promptly, data structures are never mutated concurrently.

Threads or processes would be a better fit if the system were CPU-bound, for example if each reading required heavy image processing or cryptographic hashing. That is not the case here – the only CPU work is Protobuf serialisation and simple JSON generation, which are very fast.

D. HTTP mechanics

Two HTTP features are central to the design.

Content negotiation (via the Accept header) allows the same REST endpoint to serve JSON, XML, or YAML without changing the URL. The server inspects the Accept header of each request, ranks the client's preferences against the supported media types, and selects the best match (falling back to JSON if none match). This decouples the resource identifier from the representation, so clients that understand different formats can coexist without the server maintaining separate endpoints (e.g., /sensors.json vs /sensors.xml). The implementation uses the parse_accept_header function from aiohttp to handle quality values (q=0.x).

Cookies are used to identify a client session. The first request from a client receives a Set-Cookie header containing a randomly generated session ID. The client must echo that cookie in subsequent requests. This session allows the server to associate multiple requests with the same logical client, useful for future features such as per-client rate limiting or personalised dashboards. In the current design, the cookie is simply verified to exist; no server-side session state is stored, keeping the REST API stateless while still being able to differentiate clients in logs and metrics. The cookie is set using aiohttp's set_cookie method and read from request.cookies.

Neither persistent connections nor the WebSocket upgrade handshake were chosen for the theory paper because they are less central to the REST API's design. Persistent connections (HTTP keep-alive) are automatically handled by aiohttp, and the WebSocket upgrade is a separate end-point (/live) that follows the standard RFC 6455 handshake.

References

[1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, UC Irvine.

[2] Google. (2024). Protocol Buffers – Developer Guide. https://protobuf.dev/overview/

[3] The Python Software Foundation. (2024). asyncio – Asynchronous I/O. https://docs.python.org/3/library/asyncio.html